Software Engineering Final Exam (Part 1)

Name_____ ID_____ Signature_____

*Instructions.

1. There are 12 questions.
2. Print your answers. Unreadable handwriting will not be graded.
3. Please turn in this exam paper when you leave the room.
4. If you cheat, you will receive a zero for the exam and disciplinary actions will be taken.

1. *Specify which of the following statements are functional requirements and which are nonfunctional Requirements: (10 pts)*

   1) *"The ticket distributor must enable a traveler to buy weekly passes."*

   functional

   2) *"The ticket distributor must be written in Java."*

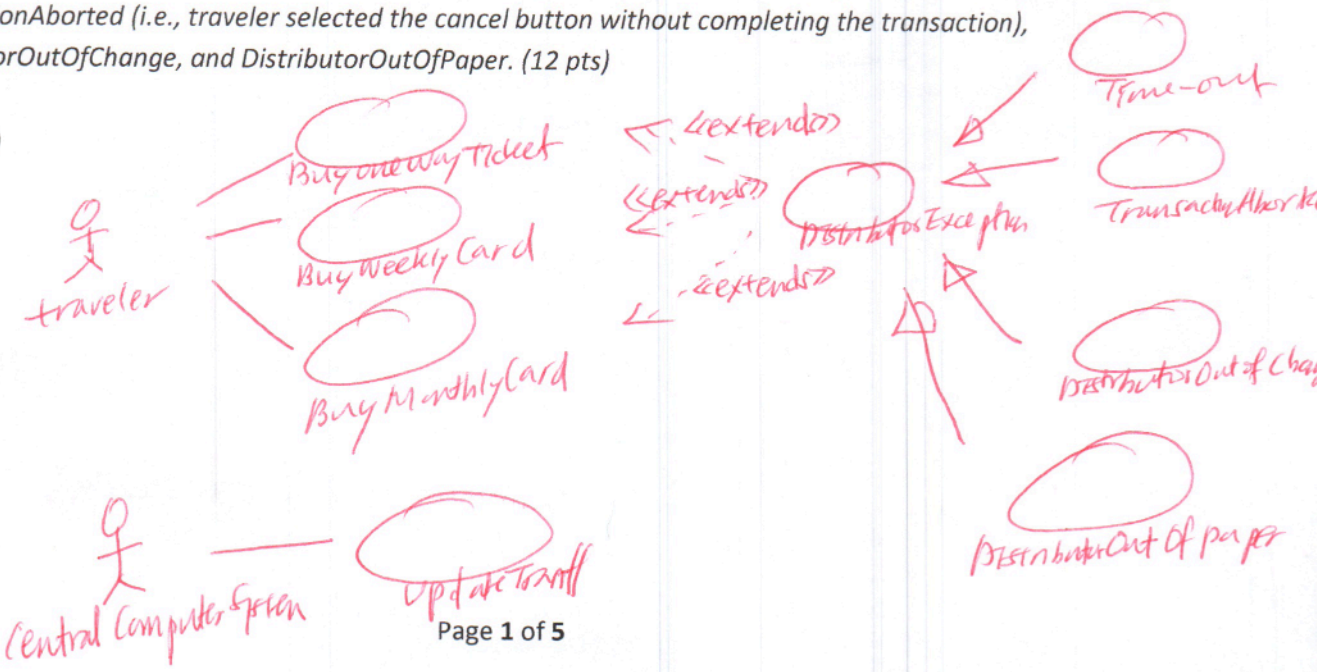
   non functional

   3) *"The ticket distributor must be easy to use.*

   non-functional

2. *Draw a use case diagram for a ticket distributor for a train system. The system includes two actors: a traveler, who purchases different types of tickets, and a central computer system, which maintains a reference database for the tariff. Use cases should include: BuyOneWayTicket, BuyWeeklyCard, BuyMonthlyCard, UpdateTariff. Also include the following exceptional cases: Time-Out (i.e., traveler took too long to insert the right amount), TransactionAborted (i.e., traveler selected the cancel button without completing the transaction), DistributorOutOfChange, and DistributorOutOfPaper. (12 pts)*

(solution)

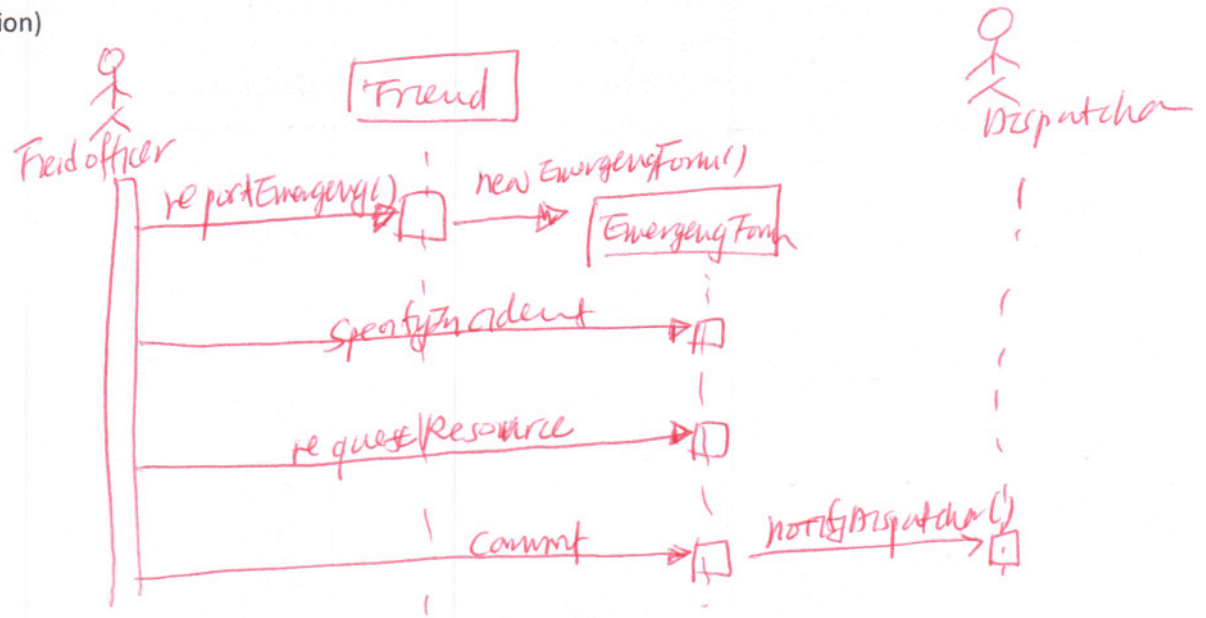


3. *Draw a sequence diagram for the ReportEmegency use case of the following figure. Draw only the first five message sends. (12 pts)*

Use case name	`ReportEmergency`
Participating actors	Initiated by `FieldOfficer` Communicates with `Dispatcher`
Flow of events	1. The `FieldOfficer` activates the "Report Emergency" function of her terminal. 2. `FRIEND` responds by presenting a form to the `FieldOfficer`. 3. The `FieldOfficer` fills out the form by selecting the emergency level, type, location, and brief description of the situation. The `FieldOfficer` also describes possible responses to the emergency situation. Once the form is completed, the `FieldOfficer` submits the form. 4. `FRIEND` receives the form and notifies the `Dispatcher`. 5. The `Dispatcher` reviews the submitted information and creates an `Incident` in the database by invoking the `OpenIncident` use case. The `Dispatcher` selects a response and acknowledges the report. 6. `FRIEND` displays the acknowledgment and the selected response to the `FieldOfficer`.
Entry condition	• The `FieldOfficer` is logged into `FRIEND`.
Exit condition	• The `FieldOfficer` has received an acknowledgment and the selected response from the `Dispatcher`, OR • The `FieldOfficer` has received an explanation indicating why the transaction could not be processed.
Quality requirements	• The `FieldOfficer`'s report is acknowledged within 30 seconds. • The selected response arrives no later than 30 seconds after it is sent by the `Dispatcher`.

An example of a use case, ReportEmergency

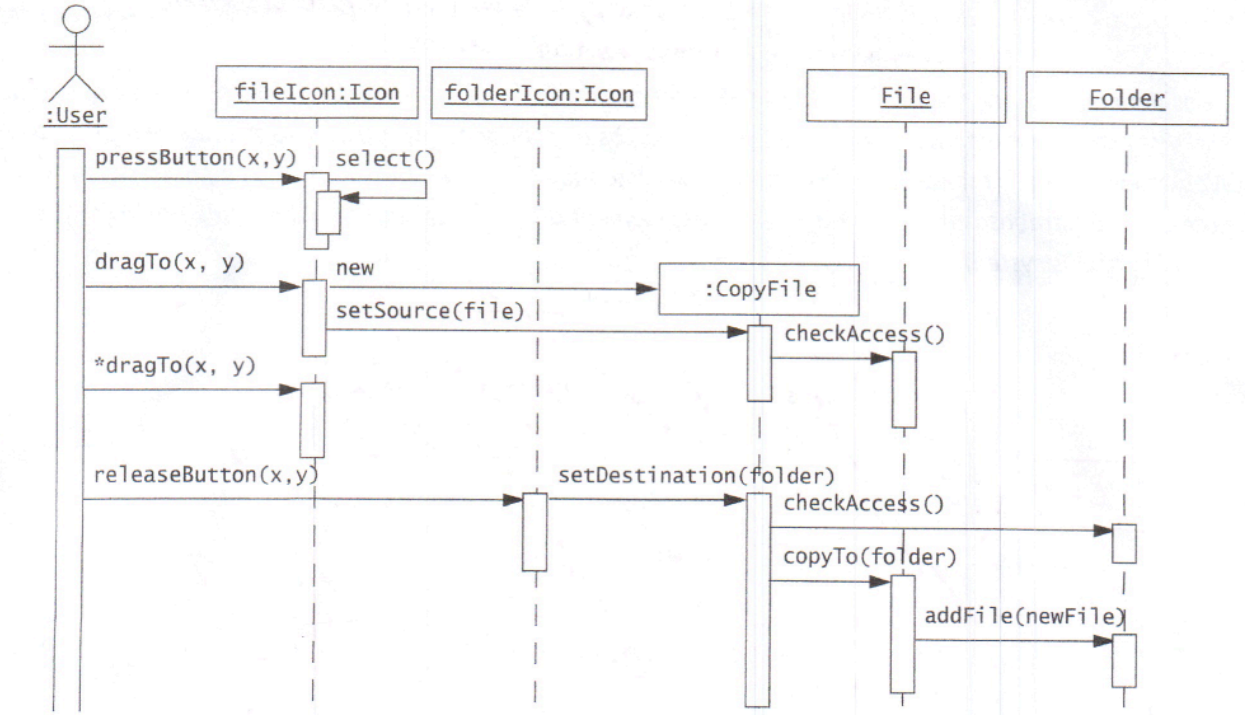
(solution)



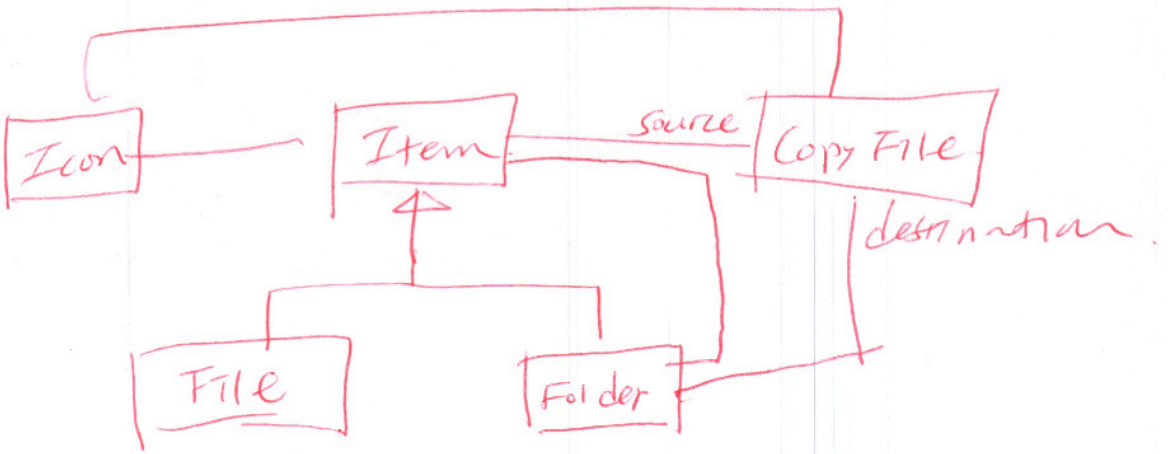
4. *What is the difference between a cross-functional team and a subsystem team? Provide examples and justify your choices. (10 pts)*

A subsystem team is completely responsible for the development of a subsystem. A cross-functional team is made out of fraction of subsystem teams and is responsible for support task, such as documentation and configuration management. Also, the system architecture is often defined by a cross-functional team made out of fraction from all subsystem teams.

5. *Examining the sequence diagram in following sequence diagram, identify the associations between these objects. These associations can then be represented on a class diagram for these objects. (12 pts)*



(solution)



6. Discuss how the MVC architecture helps or hurts the following design goals. (10 pts)

- Extensibility (e.g., the addition of new types of views).

The MVC helps Extensibility in terms of new types of views, since neither model objects nor existing view objects need to be modified to accommodate the new view.

- Response time (e.g., the time between a user input and the time all views have been updated)

A pure MVC hurts response time, because the view the user sees is updated only after the model has been updated. This can be an issue for commands involving a rapid series of interactions.

- Modifiability (e.g., the addition of new attributes in the model)

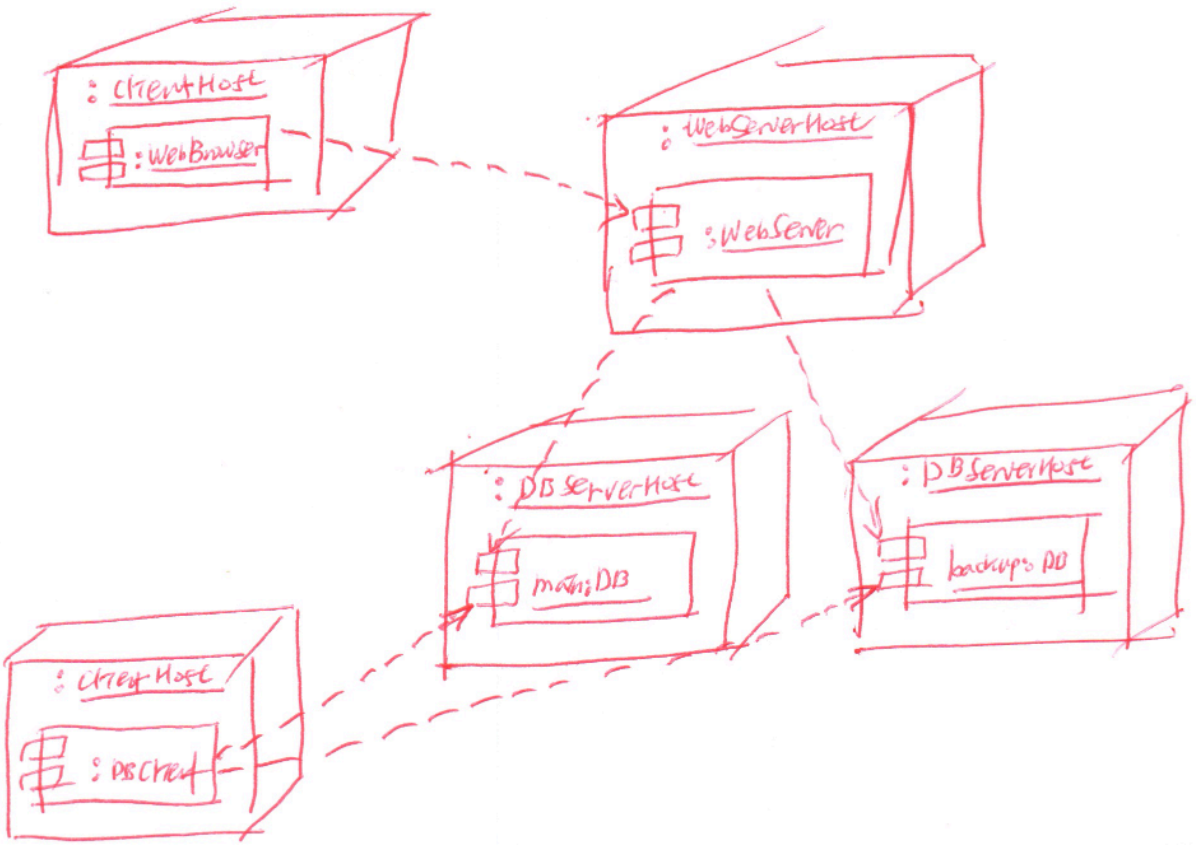
The MVC helps this goal, since only the views that need to be aware of the new attribute need to be modified. All other views can be left unchanged.

- Access control (i.e., the ability to ensure that only legitimate users can access specific parts of the model).

The MVC helps this goal, as the model is accessed using a clear interface (the public methods of the model objects) which can be controlled, for example, using a proxy pattern for each model class.

7. Consider a system that includes a Web server and two database servers. Both database servers are identical: The first acts as a main server, while the second acts as a redundant backup in case the first one fails. Users use Web browsers to access data through the Web server. They also have the option of using a proprietary client that accesses the databases directly. Draw a UML deployment diagram representing the hardware/software mapping of this system. (12 pts)

(solution)



8. *Indicate which occurrences of the following inheritance relationships are specification inheritance and which are implementation inheritance: (10 pts)*

- *A Rectangle class inherits from a Polygon class. [(* Specification *) inheritance]*
- *A Set class inherits from a BinaryTree class. [(* Implementation *) inheritance]*
- *A Set class inherits from a Bag class (a Bag is defined as an unordered collection). [(* Specification *) inheritance]*
- *A Player class inherits from a User class. [(* Specification *) inheritance]*
- *A Window class inherits from a Polygon class. [(* Implementation *) inheritance]*

9. *Consider an application that must select dynamically an encryption algorithm based on security requirements and computing time constraints. Which design pattern would you select? Draw a UML class diagram depicting the classes in the pattern and justify your choice. (12 pts)*

